

Package ‘CrossingTools’

October 4, 2025

Title Tools for Cross Optimization in Plant Breeding Programs

Version 0.0.0.9000

Description Mate allocation is a critical component of plant breeding programs, aiming to generate superior progenies and improved populations over time. While numerous methodologies for mate allocation exist, few software tools offer a unified and scalable framework that integrates these methodologies in a user-friendly manner. 'CrossingTools' addresses this gap by providing a flexible, efficient solution for optimizing mating decisions across a wide range of breeding schemes. The package supports genomic BLUP and selection indices for various variance structures and implements multiple selection criteria to evaluate cross potential, including expected mean, optimal haploid value (OHV), and superior progeny value (SPV). Different methods are provided for calculating additive and dominance segregation variances for both inbred and heterozygous parents, making 'CrossingTools' suitable for a wide range of breeding strategies. After criterion calculation, users can rank potential crosses or apply the built-in optimal crossing selection routine, which balances genetic gain and diversity to support long-term improvement. The package combines R's scripting flexibility with high-performance C++ routines via 'Rcpp' and 'RcppArmadillo', ensuring scalability for large populations with dense marker data.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

LinkingTo Rcpp,
RcppArmadillo

Imports Rcpp,
RcppArmadillo

Suggests knitr,
rmarkdown,
testthat (>= 3.0.0)

Config/testthat/edition 3

VignetteBuilder knitr

R topics documented:

calculate_optimal_haploid_value 2

calculate_variances_4W_DH	2
calculate_variances_DH	4
calculate_variances_F1	5
optimal_cross_selection	6
u_from_g	8

Index	10
--------------	-----------

calculate_optimal_haploid_value

Optimal Haplotypic Value (OHV) for proposed crosses

Description

Computes the OHV for each cross using block-wise local GEBVs. If haplotype.blocks is not supplied (or empty), each marker column of M is treated as a separate block.

Usage

```
calculate_optimal_haploid_value(crosses, haplotype.blocks, M, u, nthreads = 4L)
```

Arguments

crosses	A numeric matrix or data.frame with two columns (P1, P2), giving indices of parents (rows of M) for each proposed cross.
haplotype.blocks	A list of integer vectors. Each vector contains marker indices (columns of M) belonging to that block. If NULL or length 0, each marker is used as a separate block.
M	Numeric genotype/marker matrix (individuals x markers).
u	Numeric vector of marker effects (length ncol(M)).
nthreads	Integer ≥ 1 . OpenMP threads (if enabled at compile time).

Value

A data.frame with column OHV.

calculate_variances_4W_DH

Segregation variance for DH lines from specified four- or three-way crosses (Allier)

Description

Calculate expected genomic estimated breeding values (EGBV), segregation variance, and superior progeny value (SPV) for doubled haploid (DH) lines derived after t rounds of random mating.

Usage

```
calculate_variances_4W_DH(
  crosses,
  genetic.map,
  M,
  U,
  t,
  intensity,
  covariance,
  nThreads = 4L
)
```

Arguments

<code>crosses</code>	A data.frame or matrix (<code>n_crosses</code> x 4) of parental indices (row indices into <code>M</code>) specifying four- or three-way crosses. For three-way crosses, the first two crossing partners are the same. You can also compute the variance for two heterozygous individuals by passing haplotypes in <code>M</code> ; then indices refer to haplotypes.
<code>genetic.map</code>	A data.frame with columns: • <code>site</code>: integer marker index (1..<code>ncol(M)</code>) • <code>chr</code>: chromosome identifier • <code>pos</code>: position on the chromosome in cM All markers in <code>M</code> must appear in <code>genetic.map\$site</code> .
<code>M</code>	Numeric marker matrix (genotypes/haplotypes in rows, markers in columns) with entries in <code>c(0, 2)</code> corresponding to <code>c("AA", "BB")</code> . For heterozygotes, store haplotypes in <code>M</code> (each individual uses two rows).
<code>U</code>	Numeric matrix of marker effects (markers in rows, traits in columns). Must have <code>nrow(U) == ncol(M)</code> .
<code>t</code>	Integer. Number of random-mating generations before DH creation.
<code>intensity</code>	Numeric. Standardized selection differential.
<code>covariance</code>	Logical. If <code>TRUE</code> , also compute the segregation covariance matrix between all supplied traits.
<code>nThreads</code>	Integer (default 4). Number of OpenMP threads (if enabled at compile time).

Value

If `covariance = TRUE`, a list with element `cross_values` (a data.frame) whose columns are named `EGBV1`, `var1`, `SPV1`, `EGBV2`, If `covariance = FALSE`, a data.frame with the same column naming.

Examples

```
## Not run:
out <- calculate_variances_DH(
  crosses = matrix(c(1,2, 3,4), ncol = 4, byrow = TRUE),
  genetic.map = data.frame(site = 1:ncol(M), chr = 1, pos = seq_len(ncol(M))),
  M = M,
  U = matrix(rnorm(ncol(M)), ncol = 1),
  t = 0,
  intensity = 1.0,
```

```

    covariance = FALSE
  )

  ## End(Not run)

```

```
calculate_variances_DH
```

Segregation variance for DH lines from specified crosses

Description

Calculate expected genomic estimated breeding values (EGBV), segregation variance, and superior progeny value (SPV) for doubled haploid (DH) lines derived after *t* rounds of random mating.

Usage

```

calculate_variances_DH(
  crosses,
  genetic.map,
  M,
  U,
  t,
  intensity,
  covariance,
  method,
  nThreads = 4L
)

```

Arguments

<code>crosses</code>	A data.frame or matrix (n_crosses x 2) of parental indices (row indices into <code>M</code>) specifying the crosses.
<code>genetic.map</code>	A data.frame with columns: • <code>site</code>: integer marker index (1..<code>ncol(M)</code>) • <code>chr</code>: chromosome identifier (numeric) • <code>pos</code>: position on the chromosome in cM (numeric) All markers in <code>M</code> must appear in <code>genetic.map\$site</code> .
<code>M</code>	Numeric marker matrix (markers in columns, genotypes in rows) with entries in <code>c(0, 2)</code> corresponding to <code>c("AA", "BB")</code> .
<code>U</code>	Numeric matrix of marker effects (markers in rows, traits in columns). Must have <code>nrow(U) == ncol(M)</code> .
<code>t</code>	Integer. Number of random-mating generations before DH creation.
<code>intensity</code>	Double. Standardized selection differential.
<code>covariance</code>	Logical. If TRUE, also compute the segregation covariance matrix between all supplied traits.
<code>method</code>	Character. Which method to use; one of <code>c("osthushenrich", "lehermeier")</code> .

Value

If covariance = TRUE, a list with element cross_values (a data.frame) whose columns are named EGBV1, var1, SPV1, EGBV2, If covariance = FALSE, a data.frame with the same column naming.

Examples

```
## Not run:
out <- calculate_variances_DH(
  crosses = matrix(c(1,2, 3,4), ncol = 2, byrow = TRUE),
  genetic.map = data.frame(site = 1:ncol(M), chr = 1, pos = seq_len(ncol(M))),
  M = M,
  U = matrix(rnorm(ncol(M)), ncol = 1),
  t = 0,
  intensity = 1.0,
  covariance = FALSE,
  method = "lehermeier"
)

## End(Not run)
```

calculate_variances_F1

Additive and dominance segregation variance of families from specified crosses (Wolfe)

Description

Calculate expected genomic estimated breeding value (EGBV), expected total genetic value (ETGV), additive and dominance segregation variance, superior progeny value with additive variance (SPV), and superior progeny value including dominance variance (TSPV) for F1 families.

Usage

```
calculate_variances_F1(
  crosses,
  genetic.map,
  hap1,
  hap2,
  U,
  D,
  intensity,
  covariance,
  n_threads = 4L
)
```

Arguments

crosses	A data.frame or matrix (n_crosses x 2) of parental indices (row indices into hap1/hap2) specifying the crosses.
genetic.map	A data.frame with columns:

- site: integer marker index (1..ncol(hap1))
- chr: chromosome identifier (numeric or factor)
- pos: position on the chromosome in cM (numeric)

All markers in hap1/hap2 must appear in `genetic.map$site`.

hap1	Numeric haplotype matrix (individuals x markers) for the first haplotype, entries in <code>c(0, 1)</code> for <code>c("A", "B")</code> .
hap2	Numeric haplotype matrix (individuals x markers) for the second haplotype, entries in <code>c(0, 1)</code> for <code>c("A", "B")</code> .
U	Numeric matrix of additive marker effects (markers in rows, traits in columns). Must have <code>nrow(U) == ncol(hap1)</code> .
D	Numeric matrix of dominance marker effects (markers in rows, traits in columns). Must have <code>nrow(D) == ncol(hap1)</code> and <code>ncol(D) == ncol(U)</code> .
intensity	Double. Standardized selection differential (used for SPV).
covariance	Logical. If TRUE, also return additive and dominance segregation covariance matrices for each cross.
n_threads	Integer (default 4). Threads for the C++ routine (if compiled with OpenMP).

Value

If `covariance = FALSE`, a `data.frame` with columns `EGBV1`, `ETGV1`, `var_a1`, `SPV1`, `var_d1`, `TSPV1`, `EGBV2`, If `covariance = TRUE`, a list with:

- `cross_values`: `data.frame` as above
- `covariances`: list with elements `additive` and `dominance`, each a list of `n_trait` x `n_trait` symmetric matrices (one per cross)

Examples

```
## Not run:
# toy shapes only
crosses <- matrix(c(1,2, 3,4), ncol = 2, byrow = TRUE)
hap1 <- matrix(sample(0:1, 20, TRUE), nrow = 5) # 5 inds x 4 markers
hap2 <- matrix(sample(0:1, 20, TRUE), nrow = 5)
genetic.map <- data.frame(site = 1:ncol(hap1), chr = c(1,1,2,2), pos = c(0,10,0,10))
U <- matrix(rnorm(ncol(hap1)), ncol = 1)
D <- matrix(rnorm(ncol(hap1)), ncol = 1)
out <- calculate_variances_F1(crosses, genetic.map, hap1, hap2, U, D, intensity = 1.0, covariance = FALSE)

## End(Not run)
```

optimal_cross_selection

Optimal cross selection via a genetic algorithm

Description

Runs a GA to select a set of `ncrosses` crosses that balance expected gain (g) and genetic similarity/diversity (via G) around a target trade-off angle.

Usage

```

optimal_cross_selection(
  ncrosses,
  target.angle,
  g,
  G,
  propability.mutate = 0.01,
  nmutate = 2,
  nselect = 500,
  npop = 10000,
  max.generation = 100,
  max.iteration = 100,
  angle.penalty,
  nthreads = 4L
)

```

Arguments

<code>ncrosses</code>	Integer. Number of crosses to select.
<code>target.angle</code>	Numeric (radians). Target angle between gain and diversity.
<code>g</code>	Numeric vector of per-potential-cross scores (length must be $nInd * (nInd - 1) / 2$). Each element corresponds to one half-diallel cross.
<code>G</code>	Matrix square relationship matrix among individuals ($nInd \times nInd$).
<code>propability.mutate</code>	Numeric in $[0, 1]$. Per-solution mutation probability in GA.
<code>nmutate</code>	Integer ≥ 0 . Number of potential mutations when a mutation occurs.
<code>nselect</code>	Integer ≥ 2 . Number of solutions retained per generation in GA.
<code>npop</code>	Integer ≥ 2 . Solution population size (solutions per generation).
<code>max.generation</code>	Integer ≥ 1 . Maximum GA iterations.
<code>max.iteration</code>	Integer ≥ 1 . Early-stop if no improvement for this many solution generations.
<code>angle.penalty</code>	Numeric ≥ 0 . Penalty weight for deviating from <code>target.angle</code> .
<code>nthreads</code>	Integer ≥ 1 . number of threads to use.

Value

A list:

crossPlan `ncrosses` x 2 matrix of selected crosses.

uMax, uMin Best/achieved mean cross value

simMax, simMin Best/achieved similarity/relationship.

uBest, simBest Mean cross value and similarity of the final best solution.

angleBest, lenBest Angle and vector length of the final best solution.

Examples

```
## Not run:
nInd <- 50
G <- diag(nInd) # toy
potCross <- nInd*(nInd-1)/2
g <- rnorm(potCross)
res <- optimal_cross_selection(
  ncrosses = 20, target.angle = pi/4, g = g, G = G,
  propability.mutate = 0.01, nmutate = 2, nselect = 200,
  npop = 2000, max.generation = 200, max.iteration = 50,
  angle.penalty = 0.5, nthreads = 4L
)

## End(Not run)
```

u_from_g

Backsolve marker effects (μ) from individual effects (g)

Description

Computes marker effects $\mu = (\text{scalingFactor} * M' * G^{-1}) * g$, with optional prior/posterior variance-covariance outputs and LD-variance by groups.

Usage

```
u_from_g(
  M,
  G,
  g,
  scalingFactor,
  tol = 1e-10,
  LDvar = FALSE,
  Grouping = NULL,
  calcPriorVcov = FALSE,
  PriorVcov = NULL,
  sigmasq = NULL,
  calcPosteriorVcov = FALSE,
  PEV = NULL,
  nThreads = 4L
)
```

Arguments

M	Numeric matrix (n x p): marker/genotype matrix (individuals in rows, markers in columns).
G	Numeric matrix (n x n): relationship matrix among individuals.
g	Numeric (n x k) or length-n vector: individual effects (one or more traits).
scalingFactor	Numeric scalar multiplier.
tol	Numeric scalar tolerance for near-singularity checks in G.
LDvar	Logical; if TRUE, compute LD variance by groups.

Grouping	NULL or a data.frame with a column chr of length ncol(M) giving group membership (only used if LDvar=TRUE). If NULL with LDvar=TRUE, all markers are treated as one group. If supplied be aware that the order must match exactly that of M
calcPriorVcov	Logical; if TRUE, compute prior Vcov over marker-by-trait effects.
PriorVcov	NULL, p x p or (p*k) x (p*k) numeric matrix; see C++ docstring.
sigmasq	NULL, scalar, length-k vector, or k x k matrix specifying trait co-variance(s); required if calcPriorVcov=TRUE.
calcPosteriorVcov	Logical; if TRUE, compute posterior Vcov using PEV.
PEV	NULL or numeric matrix of size (n*k) x (n*k) (prediction error variance).
nThreads	Integer ≥ 1 ; OpenMP threads (if enabled at compile time).

Value

A list with elements matching the C++ return:

`mu_matrix` p x k matrix of marker effects.

`variance_LD` LD variance by groups (matrix if k=1, else list), or NULL if LDvar=FALSE.

`prior_Vcov` Prior Vcov matrix if requested, else NULL.

`posterior_Vcov` Posterior Vcov matrix if requested, else empty 0x0 matrix.

`group_levels` Group labels used (only when LDvar=TRUE).

Index

`calculate_optimal_haploid_value`, [2](#)
`calculate_variances_4W_DH`, [2](#)
`calculate_variances_DH`, [4](#)
`calculate_variances_F1`, [5](#)

`optimal_cross_selection`, [6](#)

`u_from_g`, [8](#)